

[A] What would be the output of the following programs:

(a) `main( )`

```
{
    char c[2] = "A" ;
    printf ( "\n%c", c[0] ) ;
    printf ( "\n%s", c ) ;
}
```

(b) `main( )`

```
{
    char s[ ] = "Get organised! learn C!!" ;
    printf ( "\n%s", &s[2] ) ;
    printf ( "\n%s", s ) ;
    printf ( "\n%s", &s ) ;
    printf ( "\n%c", s[2] ) ;
}
```

(c) `main( )`

```
{
    char s[ ] = "No two viruses work similarly" ;
    int i = 0 ;
    while ( s[i] != 0 )
    {
        printf ( "\n%c %c", s[i], *( s + i ) ) ;
        printf ( "\n%c %c", i[s], *( i + s ) ) ;
        i++ ;
    }
}
```

(d) `main( )`

```
{
    char s[ ] = "Churchgate: no church no gate" ;
    char t[25] ;
    char *ss, *tt ;
    ss = s ;
    while ( *ss != '\0' )
        *ss++ = *tt++ ;
}
```

```
        printf ( "\n%s", t ) ;
    }

(e) main()
    {
        char str1[ ] = { 'H', 'e', 'l', 'l', 'o' } ;
        char str2[ ] = "Hello" ;

        printf ( "\n%s", str1 ) ;
        printf ( "\n%s", str2 ) ;
    }

(f) main( )
    {
        printf ( 5 + "Good Morning " ) ;
    }

(g) main( )
    {
        printf ( "%c", "abcdefgh"[4] ) ;
    }

(h) main( )
    {
        printf ( "\n%d%d", sizeof ( '3' ), sizeof ( "3" ), sizeof ( 3 ) ) ;
    }
```

[B] Point out the errors, if any, in the following programs:

```
(a) main( )
    {
        char *str1 = "United" ;
        char *str2 = "Front" ;
        char *str3 ;
        str3 = strcat ( str1, str2 ) ;
        printf ( "\n%s", str3 ) ;
    }

(b) main( )
    {
```

```

    int arr[] = { 'A', 'B', 'C', 'D' };
    int i;
    for ( i = 0 ; i <= 3 ; i++ )
        printf ( "\n%d", arr[i] );
}

```

(c) `main()`

```

{
    char arr[8] = "Rhombus";
    int i;
    for ( i = 0 ; i <= 7 ; i++ )
        printf ( "\n%d", *arr );
    arr++;
}

```

[C] Fill in the blanks:

- (a) "A" is a _____ while 'A' is a _____.
- (b) A string is terminated by a _____ character, which is written as _____.
- (c) The array **char name[10]** can consist of a maximum of _____ characters.
- (d) The array elements are always stored in _____ memory locations.

[D] Attempt the following:

- (a) Which is more appropriate for reading in a multi-word string?
`gets()` `printf()` `scanf()` `puts()`
- (b) If the string "Alice in wonder land" is fed to the following **scanf()** statement, what will be the contents of the arrays **str1**, **str2**, **str3** and **str4**?

```
scanf ( "%s%s%s%s%s", str1, str2, str3, str4 );
```

- (c) Write a program that converts all lowercase characters in a given string to its equivalent uppercase character.
- (d) Write a program that extracts part of the given string from the specified position. For example, if the sting is "Working with strings is fun", then if from position 4, 4 characters are to be extracted then the program should return string as "king". Moreover, if the position from where the string is to be extracted is given and the number of characters to be extracted is 0 then the program should extract entire string from the specified position.
- (e) Write a program that converts a string like "124" to an integer 124.
- (f) Write a program that replaces two or more consecutive blanks in a string by a single blank. For example, if the input is
Grim return to the planet of apes!!
the output should be
Grim return to the planet of apes!!

Two-dimensional array, Array of pointers to strings

[E] Answer the following:

- (a) How many bytes in memory would be occupied by the following array of pointers to strings? How many bytes would be required to store the same strings, if they are stored in a two-dimensional character array?

```
char *mess[] = {  
    "Hammer and tongs",  
    "Tooth and nail",
```

```
        "Spit and polish",  
        "You and C"  
    } ;
```

- (b) Can an array of pointers to strings be used to collect strings from the keyboard? If not, why not?

[F] Attempt the following:

- (a) Write a program that uses an array of pointers to strings **str[]**. Receive two strings **str1** and **str2** and check if **str1** is embedded in any of the strings in **str[]**. If **str1** is found, then replace it with **str2**.

```
char *str[ ]={  
    "We will teach you how to...",  
    "Move a mountain",  
    "Level a building",  
    "Erase the past",  
    "Make a million",  
    "...all through C!"  
} ;
```

For example if **str1** contains "mountain" and **str2** contains "car", then the second string in **str** should get changed to "Move a car".

- (b) Write a program to sort a set of names stored in an array in alphabetical order.
- (c) Write a program to reverse the strings stored in the following array of pointers to strings:

```
char *s[ ] = {  
    "To err is human...",  
    "But to really mess things up...",  
    "One needs to know C!!"  
} ;
```

Hint: Write a function **xstrrev (string)** which should reverse the contents of one string. Call this function for reversing each string stored in **s**.

- (d) Develop a program that receives the month and year from the keyboard as integers and prints the calendar in the following format.

September 2004						
Mon	Tue	Wed	Thu	Fri	Sat	Sun
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

Note that according to the Gregorian calendar 01/01/1900 was Monday. With this as the base the calendar should be generated.

- (e) Modify the above program suitably so that once the calendar for a particular month and year has been displayed on the

screen, then using arrow keys the user must be able to change the calendar in the following manner:

Up arrow key : Next year, same month
Down arrow key : Previous year, same month
Right arrow key : Same year, next month
Left arrow key : Same year, previous month

If the escape key is hit then the procedure should stop.

Hint: Use the **getkey()** function discussed in Chapter 8, problem number [L](c).

- (f) A factory has 3 division and stocks 4 categories of products. An inventory table is updated for each division and for each product as they are received. There are three independent suppliers of products to the factory:
 - (a) Design a data format to represent each transaction.
 - (b) Write a program to take a transaction and update the inventory.
 - (c) If the cost per item is also given write a program to calculate the total inventory values.
- (g) A dequeue is an ordered set of elements in which elements may be inserted or retrieved from either end. Using an array simulate a dequeue of characters and the operations retrieve left, retrieve right, insert left, insert right. Exceptional conditions such as dequeue full or empty should be indicated. Two pointers (namely, left and right) are needed in this simulation.
- (h) Write a program to delete all vowels from a sentence. Assume that the sentence is not more than 80 characters long.
- (i) Write a program that will read a line and delete from it all occurrences of the word 'the'.

- (j) Write a program that takes a set of names of individuals and abbreviates the first, middle and other names except the last name by their first letter.
- (k) Write a program to count the number of occurrences of any two vowels in succession in a line of text. For example, in the sentence
“Pleases read this application and give me gratuity”
such occurrences are ea, ea, ui.

10 Structures

- Why Use Structures
 - Declaring a Structure
 - Accessing Structure Elements
 - How Structure Elements are Stored
- Array of Structures
- Additional Features of Structures
- Uses of Structures
- Summary
- Exercise

Which mechanic is good enough who knows how to repair only one type of vehicle? None. Same thing is true about C language. It wouldn't have been so popular had it been able to handle only all **ints**, or all **floats** or all **chars** at a time. In fact when we handle real world data, we don't usually deal with little atoms of information by themselves—things like integers, characters and such. Instead we deal with entities that are collections of things, each thing having its own attributes, just as the entity we call a 'book' is a collection of things such as title, author, call number, publisher, number of pages, date of publication, etc. As you can see all this data is dissimilar, for example author is a string, whereas number of pages is an integer. For dealing with such collections, C provides a data type called 'structure'. A structure gathers together, different atoms of information that comprise a given entity. And structure is the topic of this chapter.

Why Use Structures

We have seen earlier how ordinary variables can hold one piece of information and how arrays can hold a number of pieces of information of the same data type. These two data types can handle a great variety of situations. But quite often we deal with entities that are collection of dissimilar data types.

For example, suppose you want to store data about a book. You might want to store its name (a string), its price (a float) and number of pages in it (an int). If data about say 3 such books is to be stored, then we can follow two approaches:

- (a) Construct individual arrays, one for storing names, another for storing prices and still another for storing number of pages.
- (b) Use a structure variable.

Let us examine these two approaches one by one. For the sake of programming convenience assume that the names of books would

be single character long. Let us begin with a program that uses arrays.

```
main()
{
    char name[3];
    float price[3];
    int pages[3], i;

    printf ( "\nEnter names, prices and no. of pages of 3 books\n" );

    for ( i = 0 ; i <= 2 ; i++ )
        scanf ( "%c %f %d", &name[i], &price[i], &pages[i] );

    printf ( "\nAnd this is what you entered\n" );
    for ( i = 0 ; i <= 2 ; i++ )
        printf ( "%c %f %d\n", name[i], price[i], pages[i] );
}
```

And here is the sample run...

Enter names, prices and no. of pages of 3 books

A 100.00 354

C 256.50 682

F 233.70 512

And this is what you entered

A 100.000000 354

C 256.500000 682

F 233.700000 512

This approach no doubt allows you to store names, prices and number of pages. But as you must have realized, it is an unwieldy approach that obscures the fact that you are dealing with a group of characteristics related to a single entity—the book.

The program becomes more difficult to handle as the number of items relating to the book go on increasing. For example, we would be required to use a number of arrays, if we also decide to store name of the publisher, date of purchase of book, etc. To solve this problem, C provides a special data type—the structure.

A structure contains a number of data types grouped together. These data types may or may not be of the same type. The following example illustrates the use of this data type.

```
main()
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };
    struct book b1, b2, b3 ;

    printf ( "\nEnter names, prices & no. of pages of 3 books\n" );
    scanf ( "%c %f %d", &b1.name, &b1.price, &b1.pages );
    scanf ( "%c %f %d", &b2.name, &b2.price, &b2.pages );
    scanf ( "%c %f %d", &b3.name, &b3.price, &b3.pages );

    printf ( "\nAnd this is what you entered" );
    printf ( "\n%c %f %d", b1.name, b1.price, b1.pages );
    printf ( "\n%c %f %d", b2.name, b2.price, b2.pages );
    printf ( "\n%c %f %d", b3.name, b3.price, b3.pages );
}
```

And here is the output...

```
Enter names, prices and no. of pages of 3 books
A 100.00 354
C 256.50 682
F 233.70 512
```

And this is what you entered

A 100.000000 354

C 256.500000 682

F 233.700000 512

This program demonstrates two fundamental aspects of structures:

- (a) declaration of a structure
- (b) accessing of structure elements

Let us now look at these concepts one by one.

Declaring a Structure

In our example program, the following statement declares the structure type:

```
struct book
{
    char name ;
    float price ;
    int pages ;
};
```

This statement defines a new data type called **struct book**. Each variable of this data type will consist of a character variable called **name**, a float variable called **price** and an integer variable called **pages**. The general form of a structure declaration statement is given below:

```
struct <structure name>
{
    structure element 1 ;
    structure element 2 ;
    structure element 3 ;
    .....
    .....
```

```
};
```

Once the new structure data type has been defined one or more variables can be declared to be of that type. For example the variables **b1**, **b2**, **b3** can be declared to be of the type **struct book**, as,

```
struct book b1, b2, b3 ;
```

This statement sets aside space in memory. It makes available space to hold all the elements in the structure—in this case, 7 bytes—one for **name**, four for **price** and two for **pages**. These bytes are always in adjacent memory locations.

If we so desire, we can combine the declaration of the structure type and the structure variables in one statement.

For example,

```
struct book
{
    char name ;
    float price ;
    int pages ;
};
struct book b1, b2, b3 ;
```

is same as...

```
struct book
{
    char name ;
    float price ;
    int pages ;
} b1, b2, b3 ;
or even...
```

```
struct
```

```
{
    char name ;
    float price ;
    int pages ;
} b1, b2, b3 ;
```

Like primary variables and arrays, structure variables can also be initialized where they are declared. The format used is quite similar to that used to initiate arrays.

```
struct book
{
    char name[10] ;
    float price ;
    int pages ;
};
struct book b1 = { "Basic", 130.00, 550 } ;
struct book b2 = { "Physics", 150.80, 800 } ;
```

Note the following points while declaring a structure type:

- (a) The closing brace in the structure type declaration must be followed by a semicolon.
- (b) It is important to understand that a structure type declaration does not tell the compiler to reserve any space in memory. All a structure declaration does is, it defines the ‘form’ of the structure.
- (c) Usually structure type declaration appears at the top of the source code file, before any variables or functions are defined. In very large programs they are usually put in a separate header file, and the file is included (using the preprocessor directive `#include`) in whichever program we want to use this structure type.

Accessing Structure Elements

Having declared the structure type and the structure variables, let us see how the elements of the structure can be accessed.

In arrays we can access individual elements of an array using a subscript. Structures use a different scheme. They use a dot (.) operator. So to refer to **pages** of the structure defined in our sample program we have to use,

b1.pages

Similarly, to refer to **price** we would use,

b1.price

Note that before the dot there must always be a structure variable and after the dot there must always be a structure element.

How Structure Elements are Stored

Whatever be the elements of a structure, they are always stored in contiguous memory locations. The following program would illustrate this:

```
/* Memory map of structure elements */
main()
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };
    struct book b1 = { 'B', 130.00, 550 } ;

    printf ( "\nAddress of name = %u", &b1.name ) ;
```



```
printf ( "\nAddress of price = %u", &b1.price ) ;  
printf ( "\nAddress of pages = %u", &b1.pages ) ;  
}
```

Here is the output of the program...

```
Address of name = 65518  
Address of price = 65519  
Address of pages = 65523
```

Actually the structure elements are stored in memory as shown in the Figure 10.1.

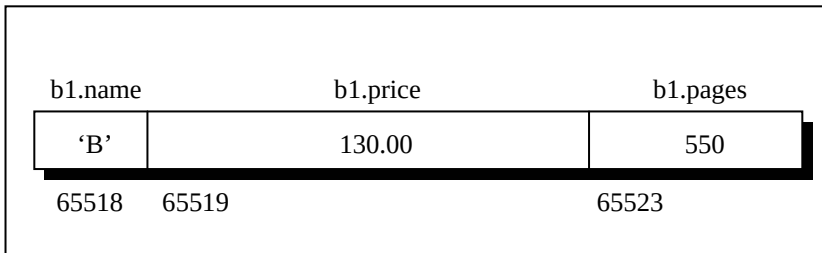


Figure 10.1

Array of Structures

Our sample program showing usage of structure is rather simple minded. All it does is, it receives values into various structure elements and output these values. But that's all we intended to do anyway... show how structure types are created, how structure variables are declared and how individual elements of a structure variable are referenced.

In our sample program, to store data of 100 books we would be required to use 100 different structure variables from **b1** to **b100**, which is definitely not very convenient. A better approach would be to use an array of structures. Following program shows how to use an array of structures.

```
/* Usage of an array of structures */
main()
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };

    struct book b[100];
    int i ;

    for ( i = 0 ; i <= 99 ; i++ )
    {
        printf ( "\nEnter name, price and pages " ) ;
        scanf ( "%c %f %d", &b[i].name, &b[i].price, &b[i].pages ) ;
    }

    for ( i = 0 ; i <= 99 ; i++ )
        printf ( "\n%c %f %d", b[i].name, b[i].price, b[i].pages ) ;
}

linkfloat()
{
    float a = 0, *b ;
    b = &a ; /* cause emulator to be linked */
    a = *b ; /* suppress the warning - variable not used */
}
```

Now a few comments about the program:

(a) Notice how the array of structures is declared...

```
struct book b[100] ;
```

This provides space in memory for 100 structures of the type **struct book**.

- (b) The syntax we use to reference each element of the array **b** is similar to the syntax used for arrays of **ints** and **chars**. For example, we refer to zeroth book's price as **b[0].price**. Similarly, we refer first book's pages as **b[1].pages**.
- (c) It should be appreciated what careful thought Dennis Ritchie has put into C language. He first defined array as a collection of similar elements; then realized that dissimilar data types that are often found in real life cannot be handled using arrays, therefore created a new data type called structure. But even using structures programming convenience could not be achieved, because a lot of variables (**b1** to **b100** for storing data about hundred books) needed to be handled. Therefore he allowed us to create an array of structures; an array of similar data types which themselves are a collection of dissimilar data types. Hats off to the genius!
- (d) In an array of structures all elements of the array are stored in adjacent memory locations. Since each element of this array is a structure, and since all structure elements are always stored in adjacent locations you can very well visualise the arrangement of array of structures in memory. In our example, **b[0]'s name, price and pages** in memory would be immediately followed by **b[1]'s name, price and pages**, and so on.
- (e) What is the function **linkfloat()** doing here? If you don't define it you are bound to get the error "Floating Point Formats Not Linked" with majority of C Compilers. What causes this error to occur? When parsing our source file, if the compiler encounters a reference to the address of a float, it sets a flag to have the linker link in the floating-point emulator. A floating point emulator is used to manipulate floating point numbers in runtime library functions like

scanf() and **atof()**. There are some cases in which the reference to the **float** is a bit obscure and the compiler does not detect the need for the emulator. The most common is using **scanf()** to read a **float** in an array of structures as shown in our program.

How can we force the formats to be linked? That's where the **linkfloat()** function comes in. It forces linking of the floating-point emulator into an application. There is no need to call this function, just define it anywhere in your program.

Additional Features of Structures

Let us now explore the intricacies of structures with a view of programming convenience. We would highlight these intricacies with suitable examples:

- (a) The values of a structure variable can be assigned to another structure variable of the same type using the assignment operator. It is not necessary to copy the structure elements piece-meal. Obviously, programmers prefer assignment to piece-meal copying. This is shown in the following example.

```
main( )
{
    struct employee
    {
        char name[10];
        int age;
        float salary;
    };
    struct employee e1 = { "Sanjay", 30, 5500.50 };
    struct employee e2, e3;

    /* piece-meal copying */
    strcpy ( e2.name, e1.name );
    e2.age = e1.age;
```

```
e2.salary = e1.salary ;

/* copying all elements at one go */
e3 = e2 ;

printf ( "\n%s %d %f", e1.name, e1.age, e1.salary ) ;
printf ( "\n%s %d %f", e2.name, e2.age, e2.salary ) ;
printf ( "\n%s %d %f", e3.name, e3.age, e3.salary ) ;
}
```

The output of the program would be...

```
Sanjay 30 5500.500000
Sanjay 30 5500.500000
Sanjay 30 5500.500000
```

Ability to copy the contents of all structure elements of one variable into the corresponding elements of another structure variable is rather surprising, since C does not allow assigning the contents of one array to another just by equating the two. As we saw earlier, for copying arrays we have to copy the contents of the array element by element.

This copying of all structure elements at one go has been possible only because the structure elements are stored in contiguous memory locations. Had this not been so, we would have been required to copy structure variables element by element. And who knows, had this been so, structures would not have become popular at all.

- (b) One structure can be nested within another structure. Using this facility complex data types can be created. The following program shows nested structures at work.

```
main()
{
    struct address
```

```
{
    char phone[15];
    char city[25];
    int pin;
};

struct emp
{
    char name[25];
    struct address a;
};

struct emp e = { "jeru", "531046", "nagpur", 10 };

printf ( "\nname = %s phone = %s", e.name, e.a.phone );
printf ( "\ncity = %s pin = %d", e.a.city, e.a.pin );
}
```

And here is the output...

```
name = jeru phone = 531046
city = nagpur pin = 10
```

Notice the method used to access the element of a structure that is part of another structure. For this the dot operator is used twice, as in the expression,

e.a.pin or e.a.city

Of course, the nesting process need not stop at this level. We can nest a structure within a structure, within another structure, which is in still another structure and so on... till the time we can comprehend the structure ourselves. Such construction however gives rise to variable names that can be surprisingly self descriptive, for example:

maruti.engine.bolt.large.qty

This clearly signifies that we are referring to the quantity of large sized bolts that fit on an engine of a maruti car.

- (c) Like an ordinary variable, a structure variable can also be passed to a function. We may either pass individual structure elements or the entire structure variable at one go. Let us examine both the approaches one by one using suitable programs.

```
/* Passing individual structure elements */
main()
{
    struct book
    {
        char name[25];
        char author[25];
        int callno;
    };
    struct book b1 = { "Let us C", "YPK", 101 };

    display ( b1.name, b1.author, b1.callno );
}

display ( char *s, char *t, int n )
{
    printf ( "\n%s %s %d", s, t, n );
}
```

And here is the output...

Let us C YPK 101

Observe that in the declaration of the structure, **name** and **author** have been declared as arrays. Therefore, when we call the function **display()** using,

```
display ( b1.name, b1.author, b1.callno );
```

we are passing the base addresses of the arrays **name** and **author**, but the value stored in **callno**. Thus, this is a mixed call—a call by reference as well as a call by value.

It can be immediately realized that to pass individual elements would become more tedious as the number of structure elements go on increasing. A better way would be to pass the entire structure variable at a time. This method is shown in the following program.

```
struct book
{
    char name[25];
    char author[25];
    int callno;
};

main()
{
    struct book b1 = { "Let us C", "YPK", 101 };
    display ( b1 );
}

display ( struct book b )
{
    printf ( "\n%s %s %d", b.name, b.author, b.callno );
}
```

And here is the output...

Let us C YPK 101

Note that here the calling of function **display()** becomes quite compact,

```
display ( b1 );
```


Having collected what is being passed to the **display()** function, the question comes, how do we define the formal arguments in the function. We cannot say,

```
struct book b1 ;
```

because the data type **struct book** is not known to the function **display()**. Therefore, it becomes necessary to define the structure type **struct book** outside **main()**, so that it becomes known to all functions in the program.

- (d) The way we can have a pointer pointing to an **int**, or a pointer pointing to a **char**, similarly we can have a pointer pointing to a **struct**. Such pointers are known as ‘structure pointers’.

Let us look at a program that demonstrates the usage of a structure pointer.

```
main()  
{  
    struct book  
    {  
        char name[25] ;  
        char author[25] ;  
        int callno ;  
    } ;  
    struct book b1 = { "Let us C", "YPK", 101 } ;  
    struct book *ptr ;  
  
    ptr = &b1 ;  
    printf ( "\n%s %s %d", b1.name, b1.author, b1.callno ) ;  
    printf ( "\n%s %s %d", ptr->name, ptr->author, ptr->callno ) ;  
}
```

The first **printf()** is as usual. The second **printf()** however is peculiar. We can't use **ptr.name** or **ptr.callno** because **ptr** is not a structure variable but a pointer to a structure, and the dot

operator requires a structure variable on its left. In such cases C provides an operator `->`, called an arrow operator to refer to the structure elements. Remember that on the left hand side of the `.'` structure operator, there must always be a structure variable, whereas on the left hand side of the `->` operator there must always be a pointer to a structure. The arrangement of the structure variable and pointer to structure in memory is shown in the Figure 10.2.

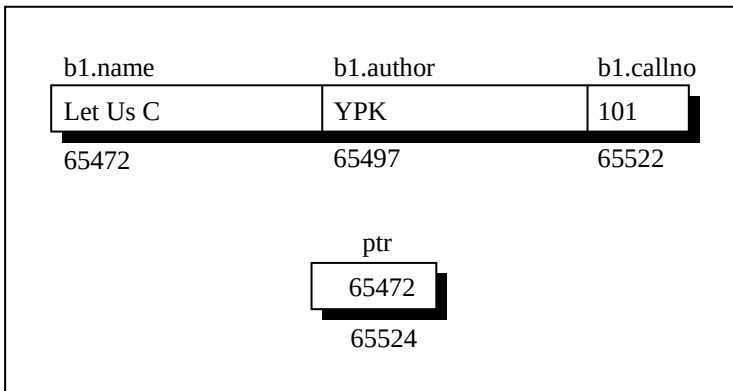


Figure 10.2

Can we not pass the address of a structure variable to a function? We can. The following program demonstrates this.

```
/* Passing address of a structure variable */
struct book
{
    char name[25];
    char author[25];
    int callno;
};

main()
{
    struct book b1 = {"Let us C", "YPK", 101};
    display (&b1);
}
```

```
}

display ( struct book *b )
{
    printf ( "\n%s %s %d", b->name, b->author, b->callno ) ;
}
```

And here is the output...

Let us C YPK 101

Again note that to access the structure elements using pointer to a structure we have to use the ‘->’ operator.

Also, the structure **struct book** should be declared outside **main()** such that this data type is available to **display()** while declaring pointer to the structure.

(e) Consider the following code snippet:

```
struct emp
{
    int a ;
    char ch ;
    float s ;
};
struct emp e ;
printf ( "%u %u %u", &e.a, &e.ch, &e.s ) ;
```

If we execute this program using TC/TC++ compiler we get the addresses as:

65518 65520 65521

As expected, in memory the **char** begins immediately after the **int** and **float** begins immediately after the **char**.

However, if we run the same program using VC++ compiler then the output turns out to be:

```
1245044 1245048 1245052
```

It can be observed from this output that the **float** doesn't get stored immediately after the **char**. In fact there is a hole of three bytes after the **char**. Let us understand the reason for this. VC++ is a 32-bit compiler targeted to generate code for a 32-bit microprocessor. The architecture of this microprocessor is such that it is able to fetch the data that is present at an address, which is a multiple of four much faster than the data present at any other address. Hence the VC++ compiler aligns every element of a structure at an address that is multiple of four. That's the reason why there were three holes created between the **char** and the **float**.

However, some programs need to exercise precise control over the memory areas where data is placed. For example, suppose we wish to read the contents of the boot sector (first sector on the floppy/hard disk) into a structure. For this the byte arrangement of the structure elements must match the arrangement of various fields in the boot sector of the disk. The **#pragma pack** directive offers a way to fulfill this requirement. This directive specifies packing alignment for structure members. The pragma takes effect at the first structure declaration after the pragma is seen. Turbo C/C++ compiler doesn't support this feature, VC++ compiler does. The following code shows how to use this directive.

```
#pragma pack(1)
struct emp
{
    int a ;
    char ch ;
    float s ;
};
```

```
#pragma pack( )  
  
struct emp e ;  
printf ( "%u %u %u", &e.a, &e.ch, &e.s ) ;
```

Here, **#pragma pack (1)** lets each structure element to begin on a 1-byte boundary as justified by the output of the program given below:

```
1245044 1245048 1245049
```

Uses of Structures

Where are structures useful? The immediate application that comes to the mind is Database Management. That is, to maintain data about employees in an organization, books in a library, items in a store, financial accounting transactions in a company etc. But mind you, use of structures stretches much beyond database management. They can be used for a variety of purposes like:

- (a) Changing the size of the cursor
- (b) Clearing the contents of the screen
- (c) Placing the cursor at an appropriate position on screen
- (d) Drawing any graphics shape on the screen
- (e) Receiving a key from the keyboard
- (f) Checking the memory size of the computer
- (g) Finding out the list of equipment attached to the computer
- (h) Formatting a floppy
- (i) Hiding a file from the directory
- (j) Displaying the directory of a disk
- (k) Sending the output to printer
- (l) Interacting with the mouse

And that is certainly a very impressive list! At least impressive enough to make you realize how important a data type a structure is and to be thorough with it if you intend to program any of the

above applications. Some of these applications would be discussed in Chapters 16 to 19.

Summary

- (a) A structure is usually used when we wish to store dissimilar data together.
- (b) Structure elements can be accessed through a structure variable using a dot (.) operator.
- (c) Structure elements can be accessed through a pointer to a structure using the arrow (->) operator.
- (d) All elements of one structure variable can be assigned to another structure variable using the assignment (=) operator.
- (e) It is possible to pass a structure variable to a function either by value or by address.
- (f) It is possible to create an array of structures.

Exercise

[A] What would be the output of the following programs:

```
(a) main()  
{  
    struct gospel  
    {  
        int num ;  
        char mess1[50] ;  
        char mess2[50] ;  
    } m ;  
  
    m.num = 1 ;  
    strcpy ( m.mess1, "If all that you have is hammer" ) ;  
    strcpy ( m.mess2, "Everything looks like a nail" ) ;  
  
    /* assume that the strucure is located at address 1004 */  
    printf ( "\n%u %u %u", &m.num, m.mess1, m.mess2 ) ;  
}
```

```
(b) struct gospel
{
    int num ;
    char mess1[50] ;
    char mess2[50] ;
} m1 = { 2, "If you are driven by success",
        "make sure that it is a quality drive"
        };
main()
{
    struct gospel m2, m3 ;
    m2 = m1 ;
    m3 = m2 ;
    printf ( "\n%d %s %s", m1.num, m2.mess1, m3.mess2 ) ;
}
```

[B] Point out the errors, if any, in the following programs:

```
(a) main()
{
    struct employee
    {
        char name[25] ;
        int age ;
        float bs ;
    };
    struct employee e ;
    strcpy ( e.name, "Hacker" ) ;
    age = 25 ;
    printf ( "\n%s %d", e.name, age ) ;
}
```

```
(b) main()
{
    struct
    {
        char name[25] ;
```

```

        char language[10];
    };
    struct employee e = { "Hacker", "C" };
    printf ( "\n%s %d", e.name, e.language );
}

(c) struct virus
{

    char signature[25];
    char status[20];
    int size;
} v[2] = {
    "Yankee Doodle", "Deadly", 1813,
    "Dark Avenger", "Killer", 1795
};

main()
{
    int i;
    for ( i = 0 ; i <= 1 ; i++ )
        printf ( "\n%s %s", v.signature, v.status );
}

(d) struct s
{
    char ch;
    int i;
    float a;
};

main()
{
    struct s var = { 'C', 100, 12.55 };
    f ( var );
    g ( &var );
}

f ( struct s v )
{
    printf ( "\n%c %d %f", v->ch, v->i, v->a );
}

```



```
g ( struct s *v )
{
    printf ( "\n%c %d %f", v.ch, v.i, v.a ) ;
}

(e) struct s
{
    int i ;
    struct s *p ;
};
main()
{
    struct s var1, var2 ;

    var1.i = 100 ;
    var2.i = 200 ;
    var1.p = &var2 ;
    var2.p = &var1 ;
    printf ( "\n%d %d", var1.p -> i, var2.p -> i ) ;
}
```

[C] Answer the following:

(a) Ten floats are to be stored in memory. What would you prefer, an array or a structure?

(b) Given the statement,

maruti.engine.bolts = 25 ;

which of the following is True?

1. structure bolts is nested within structure engine
2. structure engine is nested within structure maruti
3. structure maruti is nested within structure engine
4. structure maruti is nested within structure bolts

(c) State True or False:

1. All structure elements are stored in contiguous memory locations.

2. An array should be used to store dissimilar elements, and a structure to store similar elements.
3. In an array of structures, not only are all structures stored in contiguous memory locations, but the elements of individual structures are also stored in contiguous locations.

(d) struct time

```
{  
    int hours ;  
    int minutes ;  
    int seconds ;  
} t ;  
struct time *tt ;  
tt = &t ;
```

Looking at the above declarations, which of the following refers to **seconds** correctly:

1. tt.seconds
2. (*tt).seconds
3. time.t
4. tt -> seconds

[D] Attempt the following:

(a) Create a structure to specify data on students given below:

Roll number, Name, Department, Course, Year of joining

Assume that there are not more than 450 students in the collage.

- (a) Write a function to print names of all students who joined in a particular year.
- (b) Write a function to print the data of a student whose roll number is given.

- (b) Create a structure to specify data of customers in a bank. The data to be stored is: Account number, Name, Balance in account. Assume maximum of 200 customers in the bank.
 - (a) Write a function to print the Account number and name of each customer with balance below Rs. 100.
 - (b) If a customer request for withdrawal or deposit, it is given in the form:

Acct. no, amount, code (1 for deposit, 0 for withdrawal)

Write a program to give a message, "The balance is insufficient for the specified withdrawal".
- (c) An automobile company has serial number for engine parts starting from AA0 to FF9. The other characteristics of parts to be specified in a structure are: Year of manufacture, material and quantity manufactured.
 - (a) Specify a structure to store information corresponding to a part.
 - (b) Write a program to retrieve information on parts with serial numbers between BB1 and CC6.
- (d) A record contains name of cricketer, his age, number of test matches that he has played and the average runs that he has scored in each test match. Create an array of structure to hold records of 20 such cricketer and then write a program to read these records and arrange them in ascending order by average runs. Use the **qsort()** standard library function.
- (e) There is a structure called **employee** that holds information like employee code, name, date of joining. Write a program to create an array of the structure and enter some data into it. Then ask the user to enter current date. Display the names of those employees whose tenure is 3 or more than 3 years according to the given current date.
- (f) Write a menu driven program that depicts the working of a library. The menu options should be:

1. Add book information
2. Display book information
3. List all books of given author
4. List the title of specified book
5. List the count of books in the library
6. List the books in the order of accession number
7. Exit

Create a structure called **library** to hold accession number, title of the book, author name, price of the book, and flag indicating whether book is issued or not.

- (g) Write a program that compares two given dates. To store date use structure say **date** that contains three members namely date, month and year. If the dates are equal then display message as "Equal" otherwise "Unequal".
- (h) Linked list is a very common data structure often used to store similar data in memory. While the elements of an array occupy contiguous memory locations, those of a linked list are not constrained to be stored in adjacent location. The individual elements are stored “somewhere” in memory, rather like a family dispersed, but still bound together. The order of the elements is maintained by explicit links between them. Thus, a linked list is a collection of elements called nodes, each of which stores two item of information—an element of the list, and a link, i.e., a pointer or an address that indicates explicitly the location of the node containing the successor of this list element.

Write a program to build a linked list by adding new nodes at the beginning, at the end or in the middle of the linked list. Also write a function **display()** which display all the nodes present in the linked list.

- (i) A stack is a data structure in which addition of new element or deletion of existing element always takes place at the same

end. This end is often known as ‘top’ of stack. This situation can be compared to a stack of plates in a cafeteria where every new plate taken off the stack is also from the ‘top’ of the stack. There are several application where stack can be put to use. For example, recursion, keeping track of function calls, evaluation of expressions, etc. Write a program to implement a stack using a linked list.

- (j) Unlike a stack, in a queue the addition of new element takes place at the end (called ‘rear’ of queue) whereas deletion takes place at the other end (called ‘front’ of queue). Write a program to implement a queue using a linked list.

11 Console Input/Output

- Types of I/O
- Console I/O Functions
 - Formatted Console I/O Functions
 - sprintf*() and *sscanf*() Functions
 - Unformatted Console I/O Functions
- Summary
- Exercise

As mentioned in the first chapter, Dennis Ritchie wanted C to remain compact. In keeping with this intention he deliberately omitted everything related with Input/Output (I/O) from his definition of the language. Thus, C simply has no provision for receiving data from any of the input devices (like say keyboard, disk, etc.), or for sending data to the output devices (like say VDU, disk, etc.). Then how do we manage I/O, and how is it that we were able to use **printf()** and **scanf()** if C has nothing to offer for I/O? This is what we intend to explore in this chapter.

Types of I/O

Though C has no provision for I/O, it of course has to be dealt with at some point or the other. There is not much use writing a program that spends all its time telling itself a secret. Each Operating System has its own facility for inputting and outputting data from and to the files and devices. It's a simple matter for a system programmer to write a few small programs that would link the C compiler for particular Operating system's I/O facilities.

The developers of C Compilers do just that. They write several standard I/O functions and put them in libraries. These libraries are available with all C compilers. Whichever C compiler you are using it's almost certain that you have access to a library of I/O functions.

Do understand that the I/O facilities with different operating systems would be different. Thus, the way one OS displays output on screen may be different than the way another OS does it. For example, the standard library function **printf()** for DOS-based C compiler has been written keeping in mind the way DOS outputs characters to screen. Similarly, the **printf()** function for a Unix-based compiler has been written keeping in mind the way Unix outputs characters to screen. We as programmers do not have to bother about which **printf()** has been written in what manner. We should just use **printf()** and it would take care of the rest of the

details that are OS dependent. Same is true about all other standard library functions available for I/O.

There are numerous library functions available for I/O. These can be classified into three broad categories:

- (a) Console I/O functions - Functions to receive input from keyboard and write output to VDU.
- (b) File I/O functions - Functions to perform I/O operations on a floppy disk or hard disk.

In this chapter we would be discussing only Console I/O functions. File I/O functions would be discussed in Chapter 12.

Console I/O Functions

The screen and keyboard together are called a console. Console I/O functions can be further classified into two categories—formatted and unformatted console I/O functions. The basic difference between them is that the formatted functions allow the input read from the keyboard or the output displayed on the VDU to be formatted as per our requirements. For example, if values of average marks and percentage marks are to be displayed on the screen, then the details like where this output would appear on the screen, how many spaces would be present between the two values, the number of places after the decimal points, etc. can be controlled using formatted functions. The functions available under each of these two categories are shown in Figure 11.1. Now let us discuss these console I/O functions in detail.

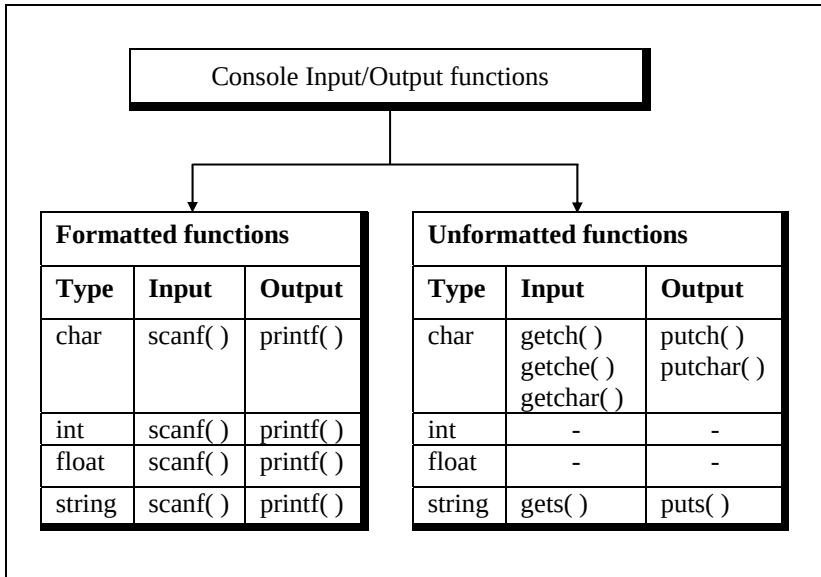


Figure 11.1

Formatted Console I/O Functions

As can be seen from Figure 11.1 the functions **printf()**, and **scanf()** fall under the category of formatted console I/O functions. These functions allow us to supply the input in a fixed format and let us obtain the output in the specified form. Let us discuss these functions one by one.

We have talked a lot about **printf()**, used it regularly, but without having introduced it formally. Well, better late than never. Its general form looks like this...

```
printf ( "format string", list of variables ) ;
```

The format string can contain:

- (a) Characters that are simply printed as they are
- (b) Conversion specifications that begin with a % sign

(c) Escape sequences that begin with a \ sign

For example, look at the following program:

```
main()  
{  
    int avg = 346 ;  
    float per = 69.2 ;  
    printf ( "Average = %d\nPercentage = %f", avg, per ) ;  
}
```

The output of the program would be...

```
Average = 346  
Percentage = 69.200000
```

How does **printf()** function interpret the contents of the format string. For this it examines the format string from left to right. So long as it doesn't come across either a % or a \ it continues to dump the characters that it encounters, on to the screen. In this example **Average** = is dumped on the screen. The moment it comes across a conversion specification in the format string it picks up the first variable in the list of variables and prints its value in the specified format. In this example, the moment **%d** is met the variable **avg** is picked up and its value is printed. Similarly, when an escape sequence is met it takes the appropriate action. In this example, the moment **\n** is met it places the cursor at the beginning of the next line. This process continues till the end of format string is not reached.

Format Specifications

The **%d** and **%f** used in the **printf()** are called format specifiers. They tell **printf()** to print the value of **avg** as a decimal integer and the value of **per** as a float. Following is the list of format specifiers that can be used with the **printf()** function.

Data type		Format specifier
Integer	short signed	%d or %l
	short unsigned	%u
	long signed	%ld
	long unsigned	%lu
	unsigned hexadecimal	%x
	unsigned octal	%o
Real	float	%f
	double	%lf
Character	signed character	%c
	unsigned character	%c
String		%s

Figure 11.2

We can provide following optional specifiers in the format specifications.

Specifier	Description
dd	Digits specifying field width
.	Decimal point separating field width from precision (precision stands for the number of places after the decimal point)
dd	Digits specifying precision
-	Minus sign for left justifying the output in the specified field width

Figure 11.3

Now a short explanation about these optional format specifiers. The field-width specifier tells **printf()** how many columns on screen should be used while printing a value. For example, **%10d** says, “print the variable as a decimal integer in a field of 10 columns”. If the value to be printed happens not to fill up the entire field, the value is right justified and is padded with blanks on the left. If we include the minus sign in format specifier (as in **%-10d**), this means left justification is desired and the value will be padded with blanks on the right. Here is an example that should make this point clear.

```
main()  
{  
    int weight = 63 ;  
  
    printf ( "\nweight is %d kg", weight ) ;  
    printf ( "\nweight is %2d kg", weight ) ;  
    printf ( "\nweight is %4d kg", weight ) ;  
    printf ( "\nweight is %6d kg", weight ) ;  
    printf ( "\nweight is %-6d kg", weight ) ;  
}
```

The output of the program would look like this ...

```
Columns    0123456789012345678901234567890  
           weight  is 63 kg  
           weight  is 63 kg  
           weight  is 63 kg  
           weight  is 63 kg  
           weight  is 63 kg
```

Specifying the field width can be useful in creating tables of numeric values, as the following program demonstrates.

```
main()  
{  
    printf ( "\n%f %f %f", 5.0, 13.5, 133.9 ) ;  
}
```

```
    printf ( "\n%f %f %f", 305.0, 1200.9, 3005.3 ) ;  
}
```

And here is the output...

```
5.000000 13.500000 133.900000  
305.000000 1200.900000 3005.300000
```

Even though the numbers have been printed, the numbers have not been lined up properly and hence are hard to read. A better way would be something like this...

```
main()  
{  
    printf ( "\n%10.1f %10.1f %10.1f", 5.0, 13.5, 133.9 ) ;  
    printf ( "\n%10.1f %10.1f %10.1f", 305.0, 1200.9, 3005.3 );  
}
```

This results into a much better output...

```
01234567890123456789012345678901  
      5.0      13.5      133.9  
    305.0    1200.9    3005.3
```

The format specifiers could be used even while displaying a string of characters. The following program would clarify this point:

```
/* Formatting strings with printf( ) */  
main()  
{  
    char  firstname1[ ] = "Sandy" ;  
    char  surname1[ ] = "Malya" ;  
    char  firstname2[ ] = "AjayKumar" ;  
    char  surname2[ ] = "Gurubaxani" ;  
  
    printf ( "\n%20s%20s", firstname1, surname1 ) ;  
    printf ( "\n%20s%20s", firstname2, surname2 ) ;  
}
```

```
}
```

And here's the output...

```
01234567890123456789012345678901234567890
                Sandy                Malya
                AjayKumar            Gurubaxani
```

The format specifier **%20s** reserves 20 columns for printing a string and then prints the string in these 20 columns with right justification. This helps lining up names of different lengths properly. Obviously, the format **%-20s** would have left justified the string.

Escape Sequences

We saw earlier how the newline character, **\n**, when inserted in a **printf()**'s format string, takes the cursor to the beginning of the next line. The newline character is an 'escape sequence', so called because the backslash symbol (****) is considered as an 'escape' character—it causes an escape from the normal interpretation of a string, so that the next character is recognized as one having a special meaning.

The following example shows usage of **\n** and a new escape sequence **\t**, called 'tab'. A **\t** moves the cursor to the next tab stop. A 80-column screen usually has 10 tab stops. In other words, the screen is divided into 10 zones of 8 columns each. Printing a tab takes the cursor to the beginning of next printing zone. For example, if cursor is positioned in column 5, then printing a tab takes it to column 8.

```
main()
{
    printf ( "You\tmust\tbe\tcrazy\nto\tthat\tthis\tbook" );
}
```

And here's the output...

```

           1           2           3           4
01234567890123456789012345678901234567890
You      must      be      crazy
to       hate      this    book
```

The `\n` character causes a new line to begin following ‘crazy’. The tab and newline are probably the most commonly used escape sequences, but there are others as well. Figure 11.4 shows a complete list of these escape sequences.

Esc. Seq.	Purpose	Esc. Seq.	Purpose
<code>\n</code>	New line	<code>\t</code>	Tab
<code>\b</code>	Backspace	<code>\r</code>	Carriage return
<code>\f</code>	Form feed	<code>\a</code>	Alert
<code>\'</code>	Single quote	<code>\"</code>	Double quote
<code>\\</code>	Backslash		

Figure 11.4

The first few of these escape sequences are more or less self-explanatory. `\b` moves the cursor one position to the left of its current position. `\r` takes the cursor to the beginning of the line in which it is currently placed. `\a` alerts the user by sounding the speaker inside the computer. Form feed advances the computer stationery attached to the printer to the top of the next page. Characters that are ordinarily used as delimiters... the single quote, double quote, and the backslash can be printed by preceding them with the backslash. Thus, the statement,

```
printf ( "He said, \"Let's do it!\"" );
```


will print...

He said, "Let's do it!"

So far we have been describing **printf()**'s specification as if we are forced to use only **%d** for an integer, only **%c** for a char, only **%s** for a string and so on. This is not true at all. In fact, **printf()** uses the specification that we mention and attempts to perform the specified conversion, and does its best to produce a proper result. Sometimes the result is nonsensical, as in case when we ask it to print a string using **%d**. Sometimes the result is useful, as in the case we ask **printf()** to print ASCII value of a character using **%d**. Sometimes the result is disastrous and the entire program blows up.

The following program shows a few of these conversions, some sensible, some weird.

```
main()
{
    char ch = 'z' ;
    int i = 125 ;
    float a = 12.55 ;
    char s[] = "hello there !" ;

    printf ( "\n%c %d %f", ch, ch, ch ) ;
    printf ( "\n%s %d %f", s, s, s ) ;
    printf ( "\n%c %d %f", i, i, i ) ;
    printf ( "\n%f %d\n", a, a ) ;
}
```

And here's the output ...

```
z 122 -9362831782501783000000000000000000000000000000000000.000000
hello there ! 3280 -
9362831782501783000000000000000000000000000000000000.000000
} 125 -9362831782501783000000000000000000000000000000000000.000000
```

12.550000 0

I would leave it to you to analyze the results by yourselves. Some of the conversions you would find are quite sensible.

Let us now turn our attention to **scanf()**. **scanf()** allows us to enter data from keyboard that will be formatted in a certain way.

The general form of **scanf()** statement is as follows:

```
scanf ( "format string", list of addresses of variables ) ;
```

For example:

```
scanf ( "%d %f %c", &c, &a, &ch ) ;
```

Note that we are sending addresses of variables (addresses are obtained by using ‘&’ the ‘address of’ operator) to **scanf()** function. This is necessary because the values received from keyboard must be dropped into variables corresponding to these addresses. The values that are supplied through the keyboard must be separated by either blank(s), tab(s), or newline(s). Do not include these escape sequences in the format string.

All the format specifications that we learnt in **printf()** function are applicable to **scanf()** function as well.

sprintf() and sscanf() Functions

The **sprintf()** function works similar to the **printf()** function except for one small difference. Instead of sending the output to the screen as **printf()** does, this function writes the output to an array of characters. The following program illustrates this.

```
main()  
{
```

```
int i = 10 ;
char ch = 'A' ;
float a = 3.14 ;
char str[20] ;

printf ( "\n%d %c %f", i, ch, a ) ;
sprintf ( str, "%d %c %f", i, ch, a ) ;
printf ( "\n%s", str ) ;
}
```

In this program the **printf()** prints out the values of **i**, **ch** and **a** on the screen, whereas **sprintf()** stores these values in the character array **str**. Since the string **str** is present in memory what is written into **str** using **sprintf()** doesn't get displayed on the screen. Once **str** has been built, its contents can be displayed on the screen. In our program this was achieved by the second **printf()** statement.

The counterpart of **sprintf()** is the **sscanf()** function. It allows us to read characters from a string and to convert and store them in C variables according to specified formats. The **sscanf()** function comes in handy for in-memory conversion of characters to values. You may find it convenient to read in strings from a file and then extract values from a string by using **sscanf()**. The usage of **sscanf()** is same as **scanf()**, except that the first argument is the string from which reading is to take place.

Unformatted Console I/O Functions

There are several standard library functions available under this category—those that can deal with a single character and those that can deal with a string of characters. For openers let us look at those which handle one character at a time.

So far for input we have consistently used the **scanf()** function. However, for some situations the **scanf()** function has one glaring weakness... you need to hit the Enter key before the function can

digest what you have typed. However, we often want a function that will read a single character the instant it is typed without waiting for the Enter key to be hit. **getch()** and **getche()** are two functions which serve this purpose. These functions return the character that has been most recently typed. The 'e' in **getche()** function means it echoes (displays) the character that you typed to the screen. As against this **getch()** just returns the character that you typed without echoing it on the screen. **getchar()** works similarly and echo's the character that you typed on the screen, but unfortunately requires Enter key to be typed following the character that you typed. The difference between **getchar()** and **fgetchar()** is that the former is a macro whereas the latter is a function. Here is a sample program that illustrates the use of these functions.

```
main()  
{  
    char ch ;  
  
    printf ( "\nPress any key to continue" ) ;  
    getch( ) ; /* will not echo the character */  
  
    printf ( "\nType any character" ) ;  
    ch = getche( ) ; /* will echo the character typed */  
  
    printf ( "\nType any character" ) ;  
    getchar( ) ; /* will echo character, must be followed by enter key */  
    printf ( "\nContinue Y/N" ) ;  
    fgetchar( ) ; /* will echo character, must be followed by enter key */  
}
```

And here is a sample run of this program...

```
Press any key to continue  
Type any character B  
Type any character W  
Continue Y/N Y
```

putch() and **putchar()** form the other side of the coin. They print a character on the screen. As far as the working of **putch()** **putchar()** and **fputchar()** is concerned it's exactly same. The following program illustrates this.

```
main()
{
    char ch = 'A' ;

    putch ( ch ) ;
    putchar ( ch ) ;
    fputchar ( ch ) ;
    putch ( 'Z' ) ;
    putchar ( 'Z' ) ;
    fputchar ( 'Z' ) ;
}
```

And here is the output...

AAAZZZ

The limitation of **putch()**, **putchar()** and **fputchar()** is that they can output only one character at a time.

gets() and puts()

gets() receives a string from the keyboard. Why is it needed? Because **scanf()** function has some limitations while receiving string of characters, as the following example illustrates...

```
main()
{
    char name[50] ;

    printf ( "\nEnter name " ) ;
    scanf ( "%s", name ) ;
    printf ( "%s", name ) ;
}
```

```
}
```

And here is the output...

```
Enter name Jonty Rhodes
Jonty
```

Surprised? Where did “Rhodes” go? It never got stored in the array **name[]**, because the moment the blank was typed after “Jonty” **scanf()** assumed that the name being entered has ended. The result is that there is no way (at least not without a lot of trouble on the programmer’s part) to enter a multi-word string into a single variable (**name** in this case) using **scanf()**. The solution to this problem is to use **gets()** function. As said earlier, it gets a string from the keyboard. It is terminated when an Enter key is hit. Thus, spaces and tabs are perfectly acceptable as part of the input string. More exactly, **gets()** gets a newline (**\n**) terminated string of characters from the keyboard and replaces the **\n** with a **\0**.

The **puts()** function works exactly opposite to **gets()** function. It outputs a string to the screen.

Here is a program which illustrates the usage of these functions:

```
main()
{
    char footballer[40];

    puts ("Enter name" );
    gets ( footballer ); /* sends base address of array */
    puts ("Happy footballing!" );
    puts ( footballer );
}
```

Following is the sample output:

```
Enter name
```

```
Jonty Rhodes  
Happy footballing!  
Jonty Rhodes
```

Why did we use two **puts()** functions to print “Happy footballing!” and “Jonty Rhodes”? Because, unlike **printf()**, **puts()** can output only one string at a time. If we attempt to print two strings using **puts()**, only the first one gets printed. Similarly, unlike **scanf()**, **gets()** can be used to read only one string at a time.

Summary

- (a) There is no keyword available in C for doing input/output.
- (b) All I/O in C is done using standard library functions.
- (c) There are several functions available for performing console input/output.
- (d) The formatted console I/O functions can force the user to receive the input in a fixed format and display the output in a fixed format.
- (e) There are several format specifiers and escape sequences available to format input and output.
- (f) Unformatted console I/O functions work faster since they do not have the overheads of formatting the input or output.

Exercise

[A] What would be the output of the following programs:

```
(a)  main( )  
    {  
        char ch ;  
        ch = getchar( ) ;  
        if ( islower ( ch ) )  
            putchar ( toupper ( ch ) ) ;  
        else  
            putchar ( tolower ( ch ) ) ;
```

```
    }  
(b)  main()  
    {  
        int i = 2 ;  
        float f = 2.5367 ;  
        char str[ ] = "Life is like that" ;  
  
        printf ( "\n%4d\t%3.3ft%4s", i, f, str ) ;  
    }  
(c)  main()  
    {  
        printf ( "More often than \b\b not \rthe person who \\  
                wins is the one who thinks he can!" ) ;  
    }  
(d)  char p[ ] = "The sixth sick sheikh's sixth ship is sick" ;  
    main()  
    {  
        int i = 0 ;  
        while ( p[i] != '\0' )  
        {  
            putchar ( p[i] ) ;  
            i++ ;  
        }  
    }
```

[B] Point out the errors, if any, in the following programs:

```
(a)  main()  
    {  
        int i ;  
        char a[ ] = "Hello" ;  
        while ( a != '\0' )  
        {  
            printf ( "%c", *a ) ;  
            a++ ;  
        }  
    }
```


- ```
(b) main()
 {
 double dval ;
 scanf ("%f", &dval) ;
 printf ("\nDouble Value = %lf", dval) ;
 }

(c) main()
 {
 int ival ;
 scanf ("%d\n", &n) ;
 printf ("\nInteger Value = %d", ival) ;
 }

(d) main()
 {
 char *mess[5] ;
 for (i = 0 ; i < 5 ; i++)
 scanf ("%s", mess[i]) ;
 }

(e) main()
 {
 int dd, mm, yy ;
 printf ("\nEnter day, month and year\n") ;
 scanf ("%d%d*c%d%d*c%d", &dd, &mm, &yy) ;
 printf ("The date is: %d - %d - %d", dd, mm, yy) ;
 }

(f) main()
 {
 char text ;
 sprintf (text, "%4d\t%2.2f\n%s", 12, 3.452, "Merry Go Round") ;
 printf ("\n%s", text) ;
 }

(g) main()
 {
 char buffer[50] ;
```

```
int no = 97;
double val = 2.34174 ;
char name[10] = "Shweta" ;

sprintf (buffer, "%d %lf %s", no, val, name) ;
printf ("\n%s", buffer) ;
sscanf (buffer, "%4d %2.2lf %s", &no, &val, name) ;
printf ("\n%s", buffer) ;
printf ("\n%d %lf %s", no, val, name) ;

}
```

[C] Answer the following:

(a) To receive the string "We have got the guts, you get the glory!!" in an array **char str[100]** which of the following functions would you use?

1. scanf ( "%s", str ) ;
2. gets ( str ) ;
3. getche ( str ) ;
4. fgetchar ( str ) ;

(b) Which function would you use if a single key were to be received through the keyboard?

1. scanf( )
2. gets( )
3. getche( )
4. getchar( )

(c) If an integer is to be entered through the keyboard, which function would you use?

1. scanf( )
2. gets( )
3. getche( )
4. getchar( )

- (d) If a character string is to be received through the keyboard which function would work faster?
1. `scanf( )`
  2. `gets( )`
- (e) What is the difference between **`getchar( )`**, **`fgetchar( )`**, **`getch( )`** and **`getche( )`**?
- (f) The format string of a **`printf( )`** function can contain:
1. Characters, format specifications and escape sequences
  2. Character, integers and floats
  3. Strings, integers and escape sequences
  4. Inverted commas, percentage sign and backslash character
- (g) A field-width specifier in a **`printf( )`** function:
1. Controls the margins of the program listing
  2. Specifies the maximum value of a number
  3. Controls the size of type used to print numbers
  4. Specifies how many columns will be used to print the number

**[D]** Answer the following:

- (a) Write down two functions **`xgets( )`** and **`xputs( )`** which work similar to the standard library functions **`gets( )`** and **`puts( )`**.
- (b) Write down a function **`getint( )`**, which would receive a numeric string from the keyboard, convert it to an integer number and return the integer to the calling function. A sample usage of **`getint( )`** is shown below:

```
main()
{
 int a;
```

```
 a = getint() ;
 printf ("you entered %d", a)
}
```

---

# ***12 File Input/Output***

---

- Data Organization
- File Operations
  - Opening a File
  - Reading from a File
  - Trouble in Opening a File
  - Closing the File
- Counting Characters, Tabs, Spaces, ...\_
- A File-copy Program
  - Writing to a File
- File Opening Modes
- String (line) I/O in Files
  - The Awkward Newline
- Record I/O in Files
- Text Files and Binary Files
- Record I/O Revisited
- Database Management
- Low Level Disk I/O
  - A Low Level File-copy Program
- I/O Under Windows
- Summary
- Exercise

Often it is not enough to just display the data on the screen. This is because if the data is large, only a limited amount of it can be stored in memory and only a limited amount of it can be displayed on the screen. It would be inappropriate to store this data in memory for one more reason. Memory is volatile and its contents would be lost once the program is terminated. So if we need the same data again it would have to be either entered through the keyboard again or would have to be regenerated programmatically. Obviously both these operations would be tedious. At such times it becomes necessary to store the data in a manner that can be later retrieved and displayed either in part or in whole. This medium is usually a ‘file’ on the disk. This chapter discusses how file I/O operations can be performed.

## Data Organization

Before we start doing file input/output let us first find out how data is organized on the disk. All data stored on the disk is in binary form. How this binary data is stored on the disk varies from one OS to another. However, this does not affect the C programmer since he has to use only the library functions written for the particular OS to be able to perform input/output. It is the compiler vendor’s responsibility to correctly implement these library functions by taking the help of OS. This is illustrated in Figure 12.1.

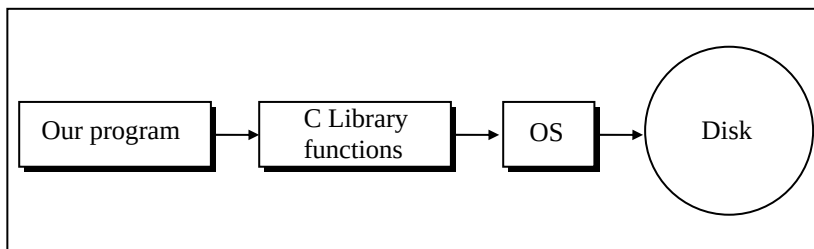


Figure 12.1

## **File Operations**

There are different operations that can be carried out on a file. These are:

- (a) Creation of a new file
- (b) Opening an existing file
- (c) Reading from a file
- (d) Writing to a file
- (e) Moving to a specific location in a file (seeking)
- (f) Closing a file

Let us now write a program to read a file and display its contents on the screen. We will first list the program and show what it does, and then dissect it line by line. Here is the listing...

```
/* Display contents of a file on screen. */
include "stdio.h"
main()
{
 FILE *fp ;
 char ch ;

 fp = fopen ("PR1.C", "r") ;

 while (1)
 {
 ch = fgetc (fp) ;

 if (ch == EOF)
 break ;

 printf ("%c", ch) ;
 }

 fclose (fp) ;
}
```

On execution of this program it displays the contents of the file 'PR1.C' on the screen. Let us now understand how it does the same.

## Opening a File

Before we can read (or write) information from (to) a file on a disk we must open the file. To open the file we have called the function **fopen( )**. It would open a file "PR1.C" in 'read' mode, which tells the C compiler that we would be reading the contents of the file. Note that "r" is a string and not a character; hence the double quotes and not single quotes. In fact **fopen( )** performs three important tasks when you open the file in "r" mode:

- (a) Firstly it searches on the disk the file to be opened.
- (b) Then it loads the file from the disk into a place in memory called buffer.
- (c) It sets up a character pointer that points to the first character of the buffer.

Why do we need a buffer at all? Imagine how inefficient it would be to actually access the disk every time we want to read a character from it. Every time we read something from a disk, it takes some time for the disk drive to position the read/write head correctly. On a floppy disk system, the drive motor has to actually start rotating the disk from a standstill position every time the disk is accessed. If this were to be done for every character we read from the disk, it would take a long time to complete the reading operation. This is where a buffer comes in. It would be more sensible to read the contents of the file into the buffer while opening the file and then read the file character by character from the buffer rather than from the disk. This is shown in Figure 12.2.



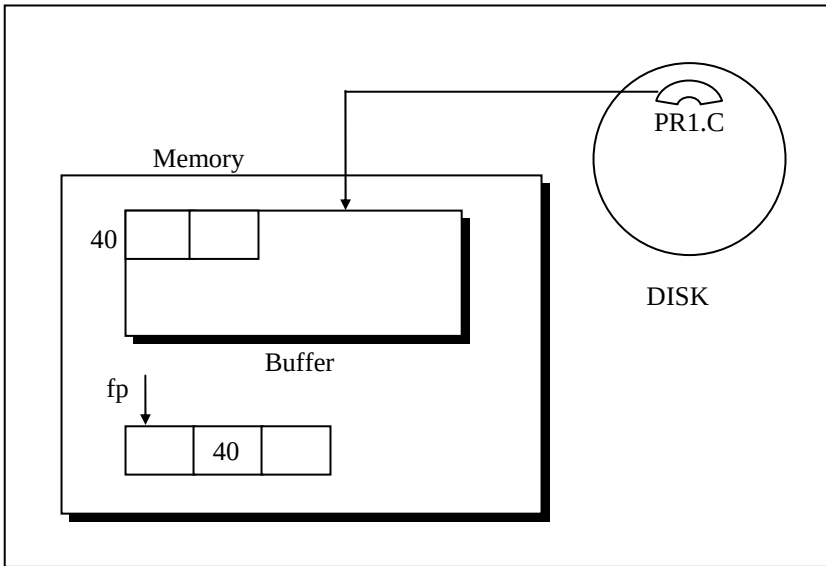


Figure 12.2

Same argument also applies to writing information in a file. Instead of writing characters in the file on the disk one character at a time it would be more efficient to write characters in a buffer and then finally transfer the contents from the buffer to the disk.

To be able to successfully read from a file information like mode of opening, size of file, place in the file from where the next read operation would be performed, etc. has to be maintained. Since all this information is inter-related, all of it is gathered together by **fopen( )** in a structure called **FILE**. **fopen( )** returns the address of this structure, which we have collected in the structure pointer called **fp**. We have declared **fp** as

```
FILE *fp ;
```

The **FILE** structure has been defined in the header file “stdio.h” (standing for standard input/output header file). Therefore, it is necessary to **#include** this file.

## Reading from a File

Once the file has been opened for reading using **fopen( )**, as we have seen, the file’s contents are brought into buffer (partly or wholly) and a pointer is set up that points to the first character in the buffer. This pointer is one of the elements of the structure to which **fp** is pointing (refer Figure 12.2).

To read the file’s contents from memory there exists a function called **fgetc( )**. This has been used in our program as,

```
ch = fgetc (fp);
```

**fgetc( )** reads the character from the current pointer position, advances the pointer position so that it now points to the next character, and returns the character that is read, which we collected in the variable **ch**. Note that once the file has been opened, we no longer refer to the file by its name, but through the file pointer **fp**.

We have used the function **fgetc( )** within an indefinite **while** loop. There has to be a way to break out of this **while**. When shall we break out... the moment we reach the end of file. But what is end of file? A special character, whose ASCII value is 26, signifies end of file. This character is inserted beyond the last character in the file, when it is created.

While reading from the file, when **fgetc( )** encounters this special character, instead of returning the character that it has read, it returns the macro EOF. The EOF macro has been defined in the file “stdio.h”. In place of the function **fgetc( )** we could have as well used the macro **getc( )** with the same effect.

In our program we go on reading each character from the file till end of file is not met. As each character is read we display it on the screen. Once out of the loop, we close the file.

## Trouble in Opening a File

There is a possibility that when we try to open a file using the function **fopen( )**, the file may not be opened. While opening the file in “r” mode, this may happen because the file being opened may not be present on the disk at all. And you obviously cannot read a file that doesn’t exist. Similarly, while opening the file for writing, **fopen( )** may fail due to a number of reasons, like, disk space may be insufficient to open a new file, or the disk may be write protected or the disk is damaged and so on.

Crux of the matter is that it is important for any program that accesses disk files to check whether a file has been opened successfully before trying to read or write to the file. If the file opening fails due to any of the several reasons mentioned above, the **fopen( )** function returns a value NULL (defined in “stdio.h” as **#define NULL 0**). Here is how this can be handled in a program...

```
#include "stdio.h"
main()
{
 FILE *fp ;

 fp = fopen ("PR1.C", "r") ;
 if (fp == NULL)
 {
 puts ("cannot open file") ;
 exit() ;
 }
}
```

## Closing the File

When we have finished reading from the file, we need to close it. This is done using the function **fclose( )** through the statement,

```
fclose (fp);
```

Once we close the file we can no longer read from it using **getc( )** unless we reopen the file. Note that to close the file we don't use the filename but the file pointer **fp**. On closing the file the buffer associated with the file is removed from memory.

In this program we have opened the file for reading. Suppose we open a file with an intention to write characters into it. This time too a buffer would get associated with it. When we attempt to write characters into this file using **fputc( )** the characters would get written to the buffer. When we close this file using **fclose( )** three operations would be performed:

- (a) The characters in the buffer would be written to the file on the disk.
- (b) At the end of file a character with ASCII value 26 would get written.
- (c) The buffer would be eliminated from memory.

You can imagine a possibility when the buffer may become full before we close the file. In such a case the buffer's contents would be written to the disk the moment it becomes full. All this buffer management is done for us by the library functions.

## Counting Characters, Tabs, Spaces, ...

Having understood the first file I/O program in detail let us now try our hand at one more. Let us write a program that will read a file and count how many characters, spaces, tabs and newlines are present in it. Here is the program...

```
/* Count chars, spaces, tabs and newlines in a file */
include "stdio.h"
main()
{
 FILE *fp ;
 char ch ;
 int nol = 0, not = 0, nob = 0, noc = 0 ;

 fp = fopen ("PR1.C", "r") ;

 while (1)
 {
 ch = fgetc (fp) ;

 if (ch == EOF)
 break ;

 noc++ ;

 if (ch == ' ')
 nob++ ;

 if (ch == '\n')
 nol++ ;

 if (ch == '\t')
 not++ ;
 }

 fclose (fp) ;
 printf ("\nNumber of characters = %d", noc) ;
 printf ("\nNumber of blanks = %d", nob) ;
 printf ("\nNumber of tabs = %d", not) ;
 printf ("\nNumber of lines = %d", nol) ;
}
```

Here is a sample run...

Number of characters = 125  
Number of blanks = 25  
Number of tabs = 13  
Number of lines = 22

The above statistics are true for a file "PR1.C", which I had on my disk. You may give any other filename and obtain different results. I believe the program is self-explanatory.

In this program too we have opened the file for reading and then read it character by character. Let us now try a program that needs to open a file for writing.

## A File-copy Program

We have already used the function **fgetc( )** which reads characters from a file. Its counterpart is a function called **fputc( )** which writes characters to a file. As a practical use of these character I/O functions we can copy the contents of one file into another, as demonstrated in the following program. This program takes the contents of a file and copies them into another file, character by character.

```
#include "stdio.h"
main()
{
 FILE *fs, *ft ;
 char ch ;

 fs = fopen ("pr1.c", "r") ;
 if (fs == NULL)
 {
 puts ("Cannot open source file") ;
 exit() ;
 }
}
```

```
 }

 ft = fopen ("pr2.c", "w");
 if (ft == NULL)
 {
 puts ("Cannot open target file");
 fclose (fs);
 exit();
 }

 while (1)
 {
 ch = fgetc (fs);

 if (ch == EOF)
 break ;
 else
 fputc (ch, ft);
 }

 fclose (fs);
 fclose (ft);
}
```

I hope most of the stuff in the program can be easily understood, since it has already been dealt with in the earlier section. What is new is only the function **fputc( )**. Let us see how it works.

## Writing to a File

The **fputc( )** function is similar to the **putch( )** function, in the sense that both output characters. However, **putch( )** function always writes to the VDU, whereas, **fputc( )** writes to the file. Which file? The file signified by **ft**. The writing process continues till all characters from the source file have been written to the target file, following which the **while** loop terminates.

Note that our sample file-copy program is capable of copying only text files. To copy files with extension .EXE or .COM, we need to open the files in binary mode, a topic that would be dealt with in sufficient detail in a later section.

## File Opening Modes

In our first program on disk I/O we have opened the file in read ("r") mode. However, "r" is but one of the several modes in which we can open a file. Following is a list of all possible modes in which a file can be opened. The tasks performed by **fopen( )** when a file is opened in each of these modes are also mentioned.

"r"      Searches file. If the file is opened successfully **fopen( )** loads it into memory and sets up a pointer which points to the first character in it. If the file cannot be opened **fopen( )** returns NULL.

Operations possible – reading from the file.

"w"      Searches file. If the file exists, its contents are overwritten. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.

Operations possible – writing to the file.

"a"      Searches file. If the file is opened successfully **fopen( )** loads it into memory and sets up a pointer that points to the last character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.

Operations possible - adding new contents at the end of file.

"r+"      Searches file. If is opened successfully **fopen( )** loads it into memory and sets up a pointer which points to the first character in it. Returns NULL, if unable to open the file.



Operations possible - reading existing contents, writing new contents, modifying existing contents of the file.

"w+" Searches file. If the file exists, its contents are overwritten. If the file doesn't exist a new file is created. Returns NULL, if unable to open file.

Operations possible - writing new contents, reading them back and modifying existing contents of the file.

"a+" Searches file. If the file is opened successfully **fopen( )** loads it into memory and sets up a pointer which points to the first character in it. If the file doesn't exist, a new file is created. Returns NULL, if unable to open file.

Operations possible - reading existing contents, appending new contents to end of file. Cannot modify existing contents.

## String (line) I/O in Files

For many purposes, character I/O is just what is needed. However, in some situations the usage of functions that read or write entire strings might turn out to be more efficient.

Reading or writing strings of characters from and to files is as easy as reading and writing individual characters. Here is a program that writes strings to a file using the function **fputs( )**.

```
/* Receives strings from keyboard and writes them to file */
#include "stdio.h"
main()
{
 FILE *fp ;
 char s[80] ;
```

```
fp = fopen ("POEM.TXT", "w") ;
if (fp == NULL)
{
 puts ("Cannot open file") ;
 exit() ;
}

printf ("\nEnter a few lines of text:\n") ;
while (strlen (gets (s)) > 0)
{
 fputs (s, fp) ;
 fputs ("\n", fp) ;
}

fclose (fp) ;
}
```

And here is a sample run of the program...

```
Enter a few lines of text:
Shining and bright, they are forever,
so true about diamonds,
more so of memories,
especially yours !
```

Note that each string is terminated by hitting enter. To terminate the execution of the program, hit enter at the beginning of a line. This creates a string of zero length, which the program recognizes as the signal to close the file and exit.

We have set up a character array to receive the string; the **fputs( )** function then writes the contents of the array to the disk. Since **fputs( )** does not automatically add a newline character to the end of the string, we must do this explicitly to make it easier to read the string back from the file.

Here is a program that reads strings from a disk file.

```
/* Reads strings from the file and displays them on screen */
#include "stdio.h"
main()
{
 FILE *fp ;
 char s[80] ;

 fp = fopen ("POEM.TXT", "r") ;
 if (fp == NULL)
 {
 puts ("Cannot open file") ;
 exit() ;
 }

 while (fgets (s, 79, fp) != NULL)
 printf ("%s", s) ;

 fclose (fp) ;
}
```

And here is the output...

Shining and bright, they are forever,  
so true about diamonds,  
more so of memories,  
especially yours !

The function **fgets( )** takes three arguments. The first is the address where the string is stored, and the second is the maximum length of the string. This argument prevents **fgets( )** from reading in too long a string and overflowing the array. The third argument, as usual, is the pointer to the structure **FILE**. When all the lines from the file have been read, we attempt to read one more line, in which case **fgets( )** returns a **NULL**.

## The Awkward Newline

We had earlier written a program that counts the total number of characters present in a file. If we use that program to count the number of characters present in the above poem (stored in the file “POEM.TXT”), it would give us the character count as 101. The same file if seen in the directory, would be reported to contain 105 characters.

This discrepancy occurs because when we attempt to write a “\n” to the file using **fputs( )**, **fputs( )** converts the \n to \r\n combination. Here \r stands for carriage return and \n for newline. If we read the same line back using **fgets( )** the reverse conversion happens. Thus when we write the first line of the poem and a “\n” using two calls to **fputs( )**, what gets written to the file is

Shining and bright, they are forever,\r\n

When the same line is read back into the array **s[ ]** using **fgets( )**, the array contains

Shining and bright, they are forever,\n\0

Thus conversion of \n to \r\n during writing and \r\n conversion to \n during reading is a feature of the standard library functions and not that of the OS. Hence the OS counts \r and \n as separate characters. In our poem there are four lines, therefore there is a discrepancy of four characters (105 - 101).

## Record I/O in Files

So far we have dealt with reading and writing only characters and strings. What if we want to read or write numbers from/to file? Furthermore, what if we desire to read/write a combination of characters, strings and numbers? For this first we would organize this dissimilar data together in a structure and then use **fprintf( )**

and **fscanf( )** library functions to read/write data from/to file. Following program illustrates the use of structures for writing records of employees.

```
/* Writes records to a file using structure */
#include "stdio.h"
main()
{
 FILE *fp ;
 char another = 'Y' ;
 struct emp
 {
 char name[40] ;
 int age ;
 float bs ;
 } ;
 struct emp e ;

 fp = fopen ("EMPLOYEE.DAT", "w") ;

 if (fp == NULL)
 {
 puts ("Cannot open file") ;
 exit() ;
 }

 while (another == 'Y')
 {
 printf ("\nEnter name, age and basic salary: ") ;
 scanf ("%s %d %f", e.name, &e.age, &e.bs) ;
 fprintf (fp, "%s %d %f\n", e.name, e.age, e.bs) ;

 printf ("Add another record (Y/N) ") ;
 fflush (stdin) ;
 another = getche() ;
 }

 fclose (fp) ;
```

```
}
```

And here is the output of the program...

```
Enter name, age and basic salary: Sunil 34 1250.50
Add another record (Y/N) Y
Enter name, age and basic salary: Sameer 21 1300.50
Add another record (Y/N) Y
Enter name, age and basic salary: Rahul 34 1400.55
Add another record (Y/N) N
```

In this program we are just reading the data into a structure variable using **scanf( )**, and then dumping it into a disk file using **fprintf( )**. The user can input as many records as he desires. The procedure ends when the user supplies 'N' for the question 'Add another record (Y/N)'.

The key to this program is the function **fprintf( )**, which writes the values in the structure variable to the file. This function is similar to **printf( )**, except that a **FILE** pointer is included as the first argument. As in **printf( )**, we can format the data in a variety of ways, by using **fprintf( )**. In fact all the format conventions of **printf( )** function work with **fprintf( )** as well.

Perhaps you are wondering what for have we used the function **fflush( )**. The reason is to get rid of a peculiarity of **scanf( )**. After supplying data for one employee, we would hit the enter key. What **scanf( )** does is it assigns name, age and salary to appropriate variables and keeps the enter key unread in the keyboard buffer. So when it's time to supply Y or N for the question 'Another employee (Y/N)', **getch( )** will read the enter key from the buffer thinking that user has entered the enter key. To avoid this problem we use the function **fflush( )**. It is designed to remove or 'flush out' any data remaining in the buffer. The argument to **fflush( )** must be the buffer which we want to flush out. Here we have used 'stdin', which means buffer related with standard input device—keyboard.

Let us now write a program that reads the employee records created by the above program. Here is how it can be done...

```
/* Read records from a file using structure */
#include "stdio.h"
main()
{
 FILE *fp ;
 struct emp
 {
 char name[40] ;
 int age ;
 float bs ;
 } ;
 struct emp e ;

 fp = fopen ("EMPLOYEE.DAT", "r") ;

 if (fp == NULL)
 {
 puts ("Cannot open file") ;
 exit() ;
 }

 while (fscanf (fp, "%s %d %f", e.name, &e.age, &e.bs) != EOF)
 printf ("\n%s %d %f", e.name, e.age, e.bs) ;

 fclose (fp) ;
}
```

And here is the output of the program...

```
Sunil 34 1250.500000
Sameer 21 1300.500000
Rahul 34 1400.500000
```

## Text Files and Binary Files

All the programs that we wrote in this chapter so far worked on text files. Some of them would not work correctly on binary files. A text file contains only textual information like alphabets, digits and special symbols. In actuality the ASCII codes of these characters are stored in text files. A good example of a text file is any C program, say PR1.C.

As against this, a binary file is merely a collection of bytes. This collection might be a compiled version of a C program (say PR1.EXE), or music data stored in a wave file or a picture stored in a graphic file. A very easy way to find out whether a file is a text file or a binary file is to open that file in Turbo C/C++. If on opening the file you can make out what is displayed then it is a text file, otherwise it is a binary file.

As mentioned while explaining the file-copy program, the program cannot copy binary files successfully. We can improve the same program to make it capable of copying text as well as binary files as shown below.

```
#include "stdio.h"
main()
{
 FILE *fs, *ft ;
 int ch ;

 fs = fopen ("pr1.exe", "rb") ;
 if (fs == NULL)
 {
 puts ("Cannot open source file") ;
 exit() ;
 }

 ft = fopen ("newpr1.exe", "wb") ;
```



```
if (ft == NULL)
{
 puts ("Cannot open target file") ;
 fclose (fs) ;
 exit() ;
}

while (1)
{
 ch = fgetc (fs) ;

 if (ch == EOF)
 break ;
 else
 fputc (ch, ft) ;
}

fclose (fs) ;
fclose (ft) ;
}
```

Using this program we can comfortably copy text as well as binary files. Note that here we have opened the source and target files in “rb” and “wb” modes respectively. While opening the file in text mode we can use either “r” or “rt”, but since text mode is the default mode we usually drop the ‘t’.

From the programming angle there are three main areas where text and binary mode files are different. These are:

- (a) Handling of newlines
- (b) Representation of end of file
- (c) Storage of numbers

Let us explore these three differences.

**Text versus Binary Mode: Newlines**

We have already seen that, in text mode, a newline character is converted into the carriage return-linefeed combination before being written to the disk. Likewise, the carriage return-linefeed combination on the disk is converted back into a newline when the file is read by a C program. However, if a file is opened in binary mode, as opposed to text mode, these conversions will not take place.

**Text versus Binary Mode: End of File**

The second difference between text and binary modes is in the way the end-of-file is detected. In text mode, a special character, whose ASCII value is 26, is inserted after the last character in the file to mark the end of file. If this character is detected at any point in the file, the read function would return the EOF signal to the program.

As against this, there is no such special character present in the binary mode files to mark the end of file. The binary mode files keep track of the end of file from the number of characters present in the directory entry of the file.

There is a moral to be derived from the end of file marker of text mode files. If a file stores numbers in binary mode, it is important that binary mode only be used for reading the numbers back, since one of the numbers we store might well be the number 26 (hexadecimal 1A). If this number is detected while we are reading the file by opening it in text mode, reading would be terminated prematurely at that point.

Thus the two modes are not compatible. See to it that the file that has been written in text mode is read back only in text mode. Similarly, the file that has been written in binary mode must be read back only in binary mode.

### Text versus Binary Mode: Storage of Numbers

The only function that is available for storing numbers in a disk file is the **fprintf( )** function. It is important to understand how numerical data is stored on the disk by **fprintf( )**. Text and characters are stored one character per byte, as we would expect. Are numbers stored as they are in memory, two bytes for an integer, four bytes for a float, and so on? No.

Numbers are stored as strings of characters. Thus, 1234, even though it occupies two bytes in memory, when transferred to the disk using **fprintf( )**, would occupy four bytes, one byte per character. Similarly, the floating-point number 1234.56 would occupy 7 bytes on disk. Thus, numbers with more digits would require more disk space.

Hence if large amount of numerical data is to be stored in a disk file, using text mode may turn out to be inefficient. The solution is to open the file in binary mode and use those functions (**fread( )** and **fwrite( )** which are discussed later) which store the numbers in binary format. It means each number would occupy same number of bytes on disk as it occupies in memory.

## Record I/O Revisited

The record I/O program that we did in an earlier section has two disadvantages:

- (a) The numbers (basic salary) would occupy more number of bytes, since the file has been opened in text mode. This is because when the file is opened in text mode, each number is stored as a character string.
- (b) If the number of fields in the structure increase (say, by adding address, house rent allowance etc.), writing structures

using **fprintf( )**, or reading them using **fscanf( )**, becomes quite clumsy.

Let us now see a more efficient way of reading/writing records (structures). This makes use of two functions **fread( )** and **fwrite( )**. We will write two programs, first one would write records to the file and the second would read these records from the file and display them on the screen.

```
/* Receives records from keyboard and writes them to a file in binary mode */
#include "stdio.h"
main()
{
 FILE *fp ;
 char another = 'Y' ;
 struct emp
 {
 char name[40] ;
 int age ;
 float bs ;
 } ;
 struct emp e ;

 fp = fopen ("EMP.DAT", "wb") ;

 if (fp == NULL)
 {
 puts ("Cannot open file") ;
 exit() ;
 }

 while (another == 'Y')
 {
 printf ("\nEnter name, age and basic salary: ") ;
 scanf ("%s %d %f", e.name, &e.age, &e.bs) ;
 fwrite (&e, sizeof (e), 1, fp) ;

 printf ("Add another record (Y/N) ") ;
```

```
 fflush (stdin) ;
 another = getche() ;
 }

 fclose (fp) ;
}
```

And here is the output...

```
Enter name, age and basic salary: Suresh 24 1250.50
Add another record (Y/N) Y
Enter name, age and basic salary: Ranjan 21 1300.60
Add another record (Y/N) Y
Enter name, age and basic salary: Harish 28 1400.70
Add another record (Y/N) N
```

Most of this program is similar to the one that we wrote earlier, which used **fprintf( )** instead of **fwrite( )**. Note, however, that the file “EMP.DAT” has now been opened in binary mode.

The information obtained from the keyboard about the employee is placed in the structure variable **e**. Then, the following statement writes the structure to the file:

```
fwrite (&e, sizeof (e), 1, fp) ;
```

Here, the first argument is the address of the structure to be written to the disk.

The second argument is the size of the structure in bytes. Instead of counting the bytes occupied by the structure ourselves, we let the program do it for us by using the **sizeof( )** operator. The **sizeof( )** operator gives the size of the variable in bytes. This keeps the program unchanged in event of change in the elements of the structure.

The third argument is the number of such structures that we want to write at one time. In this case, we want to write only one structure at a time. Had we had an array of structures, for example, we might have wanted to write the entire array at once.

The last argument is the pointer to the file we want to write to.

Now, let us write a program to read back the records written to the disk by the previous program.

```
/* Reads records from binary file and displays them on VDU */
#include "stdio.h"
main()
{
 FILE *fp ;
 struct emp
 {
 char name[40] ;
 int age ;
 float bs ;
 } ;
 struct emp e ;

 fp = fopen ("EMP.DAT", "rb") ;

 if (fp == NULL)
 {
 puts ("Cannot open file") ;
 exit() ;
 }

 while (fread (&e, sizeof (e), 1, fp) == 1)
 printf ("\n%s %d %f", e.name, e.age, e.bs) ;

 fclose (fp) ;
}
```

Here, the **fread( )** function causes the data read from the disk to be placed in the structure variable **e**. The format of **fread( )** is same as that of **fwrite( )**. The function **fread( )** returns the number of records read. Ordinarily, this should correspond to the third argument, the number of records we asked for... 1 in this case. If we have reached the end of file, since **fread( )** cannot read anything, it returns a 0. By testing for this situation, we know when to stop reading.

As you can now appreciate, any database management application in C must make use of **fread( )** and **fwrite( )** functions, since they store numbers more efficiently, and make writing/reading of structures quite easy. Note that even if the number of elements belonging to the structure increases, the format of **fread( )** and **fwrite( )** remains same.

## Database Management

So far we have learnt record I/O in bits and pieces. However, in any serious database management application, we will have to combine all that we have learnt in a proper manner to make sense. I have attempted to do this in the following menu driven program. There is a provision to Add, Modify, List and Delete records, the operations that are imperative in any database management. Following comments would help you in understanding the program easily:

- Addition of records must always take place at the end of existing records in the file, much in the same way you would add new records in a register manually.
- Listing records means displaying the existing records on the screen. Naturally, records should be listed from first record to last record.
- While modifying records, first we must ask the user which record he intends to modify. Instead of asking the record

number to be modified, it would be more meaningful to ask for the name of the employee whose record is to be modified. On modifying the record, the existing record gets overwritten by the new record.

- In deleting records, except for the record to be deleted, rest of the records must first be written to a temporary file, then the original file must be deleted, and the temporary file must be renamed back to original.
- Observe carefully the way the file has been opened, first for reading & writing, and if this fails (the first time you run this program it would certainly fail, because that time the file is not existing), for writing and reading. It is imperative that the file should be opened in binary mode.
- Note that the file is being opened only once and closed only once, which is quite logical.
- **clrscr( )** function clears the contents of the screen and **gotoxy( )** places the cursor at appropriate position on the screen. The parameters passed to **gotoxy( )** are column number followed by row number.

Given below is the complete listing of the program.

```
/* A menu-driven program for elementary database management */
#include "stdio.h"
main()
{
 FILE *fp, *ft ;
 char another, choice ;
 struct emp
 {
 char name[40] ;
 int age ;
 float bs ;
 } ;
```



```
struct emp e ;
char empname[40] ;
long int recsize ;

fp = fopen ("EMP.DAT", "rb+") ;

if (fp == NULL)
{
 fp = fopen ("EMP.DAT", "wb+") ;

 if (fp == NULL)
 {
 puts ("Cannot open file") ;
 exit() ;
 }
}

recsize = sizeof (e) ;

while (1)
{
 clrscr() ;

 gotoxy (30, 10) ;
 printf ("1. Add Records") ;
 gotoxy (30, 12) ;
 printf ("2. List Records") ;
 gotoxy (30, 14) ;
 printf ("3. Modify Records") ;
 gotoxy (30, 16) ;
 printf ("4. Delete Records") ;
 gotoxy (30, 18) ;
 printf ("0. Exit") ;
 gotoxy (30, 20) ;
 printf ("Your choice") ;

 fflush (stdin) ;
 choice = getche() ;
```

```
switch (choice)
{
 case '1' :

 fseek (fp, 0 , SEEK_END) ;
 another = 'Y' ;

 while (another == 'Y')
 {
 printf ("\nEnter name, age and basic sal. ") ;
 scanf ("%s %d %f", e.name, &e.age, &e.bs) ;
 fwrite (&e, recsize, 1, fp) ;
 printf ("\nAdd another Record (Y/N) ") ;
 fflush (stdin) ;
 another = getche () ;
 }

 break ;

 case '2' :

 rewind (fp) ;

 while (fread (&e, recsize, 1, fp) == 1)
 printf ("\n%s %d %f", e.name, e.age, e.bs) ;

 break ;

 case '3' :

 another = 'Y' ;
 while (another == 'Y')
 {
 printf ("\nEnter name of employee to modify ") ;
 scanf ("%s", empname) ;

 rewind (fp) ;
 while (fread (&e, recsize, 1, fp) == 1)
```

```
 {
 if (strcmp (e.name, empname) == 0)
 {
 printf ("\nEnter new name, age & bs");
 scanf ("%s %d %f", e.name, &e.age,
 &e.bs);
 fseek (fp, - reccount, SEEK_CUR);
 fwrite (&e, reccount, 1, fp);
 break ;
 }
 }

 printf ("\nModify another Record (Y/N) ");
 fflush (stdin);
 another = getche();
 }

 break ;

case '4' :

 another = 'Y' ;
 while (another == 'Y')
 {
 printf ("\nEnter name of employee to delete ");
 scanf ("%s", empname);

 ft = fopen ("TEMP.DAT", "wb");

 rewind (fp);
 while (fread (&e, reccount, 1, fp) == 1)
 {
 if (strcmp (e.name, empname) != 0)
 fwrite (&e, reccount, 1, ft);
 }

 fclose (fp);
 fclose (ft);
 }
}
```

```

 remove ("EMP.DAT");
 rename ("TEMP.DAT", "EMP.DAT");

 fp = fopen ("EMP.DAT", "rb+");

 printf ("Delete another Record (Y/N) ");
 fflush (stdin);
 another = getche() ;
 }
 break ;

case '0' :
 fclose (fp);
 exit() ;
}
}
}

```

To understand how this program works, you need to be familiar with the concept of pointers. A pointer is initiated whenever we open a file. On opening a file a pointer is set up which points to the first record in the file. To be precise this pointer is present in the structure to which the file pointer returned by **fopen( )** points to. On using the functions **fread( )** or **fwrite( )**, the pointer moves to the beginning of the next record. On closing a file the pointer is deactivated. Note that the pointer movement is of utmost importance since **fread( )** always reads that record where the pointer is currently placed. Similarly, **fwrite( )** always writes the record where the pointer is currently placed.

The **rewind( )** function places the pointer to the beginning of the file, irrespective of where it is present right now.

The **fseek( )** function lets us move the pointer from one record to another. In the program above, to move the pointer to the previous record from its current position, we used the function,